

Contents

SYSTEM DESIGN CASE STUDY	2
Introduction & Executive Context	3
Executive Summary	3
The Problem Statement	4
Target State Architecture & Structural Design	5
Comprehensive Solution Summary	6
End-End Customer Journey	9
Transaction Data-Flows	11
Security Architecture	13
Disaster Recovery (DR) Plan	16
Architecture Tradeoffs	19
Architecture Decision Records	21
AWS Well-Architected Framework Review	24
Architecture Risks	26
Architectural Recommendations	29
Boundary Conditions for Reference Architecture	32
Appendix A: Non-Functional Requirements (NFR) Matrix	34
Appendix B: Cost Optimization & Attribution Matrix	36
Appendix C: Enterprise Security Risk Register	38
Appendix D: Enterprise RACI Matrix	40
About the Author	42

SYSTEM DESIGN CASE STUDY

Legacy SAP Hybrid Architecture on AWS

Prepared by:

Amit Kulkarni

Introduction & Executive Context

This architecture diagram illustrates the technical patterns for integrating SAP Source Systems with AWS Cloud Analytics Services, establishing a scalable blueprint to extract, transform, and load enterprise data into a modern cloud data platform. Structurally, the framework bridges transactional SAP ERP environments—whether hosted on AWS, RISE with SAP, or traditional on-premises data centers—with an AWS Analytics Landing Zone. To solve data siloing across both the SAP HANA database level and the application layer (such as S/4HANA, ECC, and BW), the architecture provides three distinct ingestion pathways tailored to different infrastructure configurations, budgets, and latency requirements. Organizations can leverage Path A for low-code, fully managed integration using AWS Managed Services like AWS AppFlow and AWS Glue; deploy Path B for instance-based, specialized replication tools like SAP Datasphere, SAP Data Services, or SAP SLT to achieve real-time change data capture; or utilize Path C to push data directly from embedded ABAP partner add-ons. Ultimately, these diverse extraction pipelines converge into an AWS Landing Zone optimized for modern data initiatives, where AWS S3 serves as the foundational data lake storage, while AWS Redshift, AWS Kinesis, and AWS RDS provide the high-performance warehousing, real-time streaming, and relational storage capabilities necessary for advanced enterprise analytics.

Executive Summary

This architecture outlines a comprehensive framework for modernizing enterprise data strategy by securely connecting SAP Source Systems to the AWS Cloud Analytics platform. By accommodating diverse SAP deployments—including on-premises, AWS-hosted, and RISE with SAP configurations—the solution eliminates data silos across both the underlying SAP HANA database and application layers like S/4HANA, ECC, and BW. It maximizes operational flexibility through three distinct ingestion pathways: fully managed, low-code pipelines via AWS AppFlow and AWS Glue; specialized, instance-based replication solutions like SAP SLT for real-time change data capture; and direct extraction using embedded ABAP add-ons. These pathways seamlessly feed into an AWS Landing Zone, leveraging AWS S3 as a scalable data lake alongside AWS Redshift, Kinesis, and RDS to deliver high-performance data warehousing, real-time

streaming, and robust analytics capabilities that drive agile, data-driven business decisions.

The Problem Statement

1. Data Silos and Accessibility Barriers

- **Isolated Legacy Environments:** Transactional data remains locked inside rigid *SAP* architectures.
- **Complex Data Extraction:** Accessing raw data requires specialized *SAP* proprietary knowledge.
- **Cross-Platform Incompatibility:** Traditional systems struggle to natively blend *SAP* data with non-*SAP* cloud data.
- **Delayed Decision Making:** Stakeholders cannot access consolidated operational insights efficiently.

2. Infrastructure Rigidities and High Overhead

- **On-Premises Resource Scaling:** On-premises hardware limitations block rapid data volume expansions.
- **High Maintenance Costs:** Dedicated teams are required to manage complex physical infrastructure.
- **System Performance Impact:** Direct database querying risks slowing down core production *ERP* systems.
- **Rigid Monolithic Designs:** Upgrading legacy storage components requires lengthy, expensive deployment cycles.

3. Real-Time Processing Bottlenecks

- **Batch-Only Limitations:** Older extraction methods only support slow, nightly batch updates.
- **Stale Analytics Reports:** Executive dashboards suffer from day-old data latency issues.
- **Lack of Instant CDC:** Systems miss real-time *Change Data Capture* capabilities for critical events.
- **Inflexible Event Streaming:** Traditional pipelines cannot stream data directly into modern messaging queues.

4. Integration Complexity and Governance Risks

- **Custom Code Overhead:** Maintaining custom *ABAP* extraction code creates technical debt.
- **Security Compliance Gaps:** Moving sensitive data between environments increases compliance risks.

- **Fragmented Tooling Ecosystem:** Multiple disconnected point solutions complicate system management.
- **Data Lineage Loss:** Tracking data transformations accurately across distinct platforms becomes difficult.

Target State Architecture & Structural Design

System Architecture Overview

This system architecture overview outlines the end-to-end data pipeline designed to seamlessly ingest, process, and store enterprise data from varied *SAP* environments into the *AWS* cloud. By establishing a decoupled relationship between the transactional *ERP* core and the analytical cloud layer, this framework ensures high availability, minimized source system impact, and rapid data democratization across the organization.

1. Unified SAP Source System Integration

- **Multi-Deployment Flexibility:** Connects natively to *SAP* instances hosted on *AWS*, *RISE with SAP*, or traditional on-premises data centers.
- **Dual-Layer Extraction:** Targets both the low-level *SAP HANA* database layer and the high-level *SAP* application layer.
- **Broad System Support:** Extracts data seamlessly from modern *S/4HANA* environments as well as legacy *ECC* and *BW* systems.
- **Isolated Source Workloads:** Minimizes operational impact on core production systems during large-scale data extractions.

2. AWS Managed Ingestion Pipelines (Path A)

- **Serverless Connectivity:** Uses *AWS AppFlow* for low-code, automated secure data transfers directly from *SAP* applications.
- **Fully Managed ETL:** Employs *AWS Glue* to catalog, clean, and transform data moving between systems without provisioning servers.
- **Reduced Operational Overhead:** Eliminates the need to patch, maintain, or manage underlying compute infrastructure for ingestion.
- **Rapid Deployment Cycles:** Facilitates quick configuration of standardized data pipelines using cloud-native connectors.

3. Specialized and Instance-Based Replication (Paths B & C)

- **Dedicated Compute Instances:** Utilizes specialized *EC2* instances to run heavy-duty partner solutions and native *SAP* middleware.

- **Real-Time Change Data Capture:** Deploys *SAP SLT* (Landscape Transformation) to replicate database changes instantly as they occur.
- **Enterprise Data Fabric:** Integrates *SAP Datasphere* and *SAP Data Services* to maintain complex metadata models and orchestrate data.
- **Embedded Application Push:** Leverages partner *ABAP add-ons* running directly inside the *SAP* system to push optimized data payloads.

4. Scalable AWS Analytics Landing Zone

- **Foundational Data Lake:** Uses *AWS S3* as secure, highly durable object storage for raw, structured, and unstructured data.
- **High-Performance Warehousing:** Routes structured data into *AWS Redshift* for complex enterprise-wide reporting and *SQL analytics*.
- **Real-Time Stream Processing:** Feeds time-sensitive event data into *AWS Kinesis* for immediate downstream consumption.
- **Relational Storage Depth:** populates *AWS RDS* instances for operational microservices, staging areas, and transactional analytical needs.

Comprehensive Solution Summary

The proposed solution delivers a highly resilient, modern, and versatile integration engine that bridges enterprise *SAP Source* Systems with cloud-native *AWS analytics* resources. By transitioning from traditional monolithic querying to an asynchronous, multi-path ingestion and replication pattern, the architecture guarantees real-time event processing, zero data loss, automated workload isolation, and scalable high-performance enterprise intelligence orchestration.

1. End-to-End Managed Data Ingestion & Extraction Ingress Framework

The platform's data entry boundary decouples and aggregates heavy data traffic arrays traveling between the core *SAP enterprise* resource planning environments and internal *AWS analytics* infrastructure.

- **Managed Serverless Ingestion Boundary (Path A):** *SAP* application actions trigger extraction pathways that route directly into *AWS AppFlow* on an event-driven or scheduled basis. This layer abstracts complex API infrastructure management, absorbs sudden transactional traffic spikes, and eliminates the heavy overhead of traditional point-to-point HTTP polling.

- **Asynchronous Enterprise Schema Transformation:** Ingested raw payloads pass immediately into *AWS Glue*, which serves as a highly scalable, serverless data integration engine. *AWS Glue* applies automated data cataloging, cleaning, and filtering rules to transform and securely route incoming *SAP data streams* without imposing down-stream dependencies.
- **High-Throughput Specialized Replication Ingress (Paths B & C):** Real-time database mutations and application-level payloads bypass standard queues via dedicated instance-based solutions. Specialized layers like *SAP SLT* utilize change data capture (CDC) to stream live delta records, while partner *ABAP add-ons* push optimized transactional updates directly from the *SAP system core*.

2. Multi-Tier Cloud-Native Compute, Integration Routing, and Core Storage Ecosystem

The core analytics ecosystem utilizes a completely decoupled, highly durable runtime topology to store high-velocity transaction logic, maintain data lake consistency, and track historical enterprise transformations.

- **Stateless Analytics Aggregation Layer:** Fully managed analytical pipelines run across dedicated, instance-based replication systems (*SAP Datasphere* and *SAP Data Services*). This infrastructure triggers instantly upon receiving source data, parsing *multi-table SAP relations*, validating schema integrity, and orchestrating complex enterprise data fabric models.
- **High-Performance Low-Latency State Persistence:** Extracted operational datasets map directly into *AWS S3*, a secure, highly durable object storage data lake. *AWS S3* securely isolates raw landing zones from processed zones, delivering predictable, highly available access for massive scale analytical read workloads.
- **High-Volume Historical Data Warehouse Topology:** Structured business tables and operational entities route from the storage lake into *AWS Redshift*. This architecture completely decouples transactional processing from business intelligence querying, providing low-latency execution for complex, enterprise-wide *SQL reporting* and deep historical analysis.

3. Centralized Governance, Failure Isolation, and Compliance Alerting Framework

This resilient architecture deploys a fault-tolerant monitoring paradigm across all ingestion paths and storage layers to enforce deep pipeline visibility, real-time workload isolation, and proactive infrastructure alerting.

- **Automated Boundary Break Detection & Isolation:** The ingestion pipeline implements automated infrastructure protection rules; if a dedicated replication connection breaks or ingestion thresholds fail, specialized fault-tolerance paths isolate the active payload to prevent core *SAP* system performance degradation.
- **Infrastructure Triage and Tier-1 Support Notification:** Errant ingestion traces, network disconnects, and dead-letter payloads trigger localized handlers across the instance and managed boundaries. These systems format the error stack and generate automated infrastructure alerts, dispatching urgent technical notifications to the *AWS Solution Support Team* for immediate automated remediation.
- **Operational Telemetry and Functional Exception Analytics:** Operational trends coming from the streaming database layer route through integrated monitoring tools to detect systemic data irregularities or batch-load failures. Verified operational anomalies trigger automated events to deliver high-level alert summaries directly to the *Business Users & Support Team* inbox.

4. Hybrid Network Topography, Stream Consumers, and High-Availability Execution Fabric

The operational infrastructure is engineered to trigger secondary, external business workflows and downstream microservices seamlessly while ensuring continuous, low-latency transaction routing across cloud boundaries.

- **Dynamic Database Stream Consumption & Formatting:** Real-time data updates captured via the *SAP SLT* change data capture layer trigger an independent background processing loop. This function serializes real-time operational deltas and streams them directly as immutable events into *AWS Kinesis*.
- **Automated Omni-Channel Downstream Orchestration:** Operational status mutations committed within the core analytics layers invoke *AWS RDS* in real time. This relational engine processes the contextual state

updates, structuring optimized data tables for operational microservices and staging environments.

- **Proactive Business Intelligence Execution Fabric:** *AWS Kinesis* and *AWS Redshift* receive the data streams to initiate real-time analytics dashboards, ad-hoc querying loops, or executive reporting workflows. This guarantees that internal corporate planning and operational insights match active source system states instantly without human intervention.

End-End Customer Journey

Step 1: Managed ERP Data Extraction and Application Ingress

The User Action: A supply chain manager completes an inventory reconciliation or modifies a high-value procurement document within the *SAP S/4HANA* user interface.

SAP Data Transformation: Instead of waiting for slow, midnight batch schedules, the *SAP application layer* packages the operational delta, table keys, and core contextual business data.

AWS AppFlow Extraction: The outbound payload hits the entry boundary of *AWS AppFlow*. *AppFlow* maintains a secure, managed connection with *SAP*, pulling the transactional data instantly over an encrypted channel while protecting *SAP* application performance.

Step 2: Automated Schema Transformation and Cataloging

AWS Glue Ingress: *AWS AppFlow* hands off the raw, structured *SAP* payload directly to *AWS Glue*, the central serverless data integration engine.

Data Cleansing & Partitioning: *AWS Glue* runs automated transformation routines to clean inconsistent formats, maps *SAP* proprietary data types to cloud-standard schemas, and strips out duplicate fields.

Data Catalog Registration: The transformed metadata is immediately registered within the *AWS Glue Data Catalog*, making the new database schema instantly discoverable for downstream cloud analytics components without altering original source formats.

Step 3: High-Volume Real-Time Change Data Capture (CDC)

Database Mutation Interception: Simultaneously at the database tier, an inventory change triggers the *SAP SLT* (Landscape Transformation Replication Server) engine running on dedicated compute instances.

Real-Time Delta Streaming: *SAP SLT* captures the exact row-level modification inside the *SAP HANA* database and streams the delta records out of the corporate network using real-time database replication loops.

AWS Kinesis Ingestion: The live database change log feeds directly into *AWS Kinesis*. *Kinesis* absorbs the high-velocity event stream, maintaining a sequential, time-ordered record of database inserts and updates.

Step 4: Storage Consolidation and Data Lake Landing Zone

AWS S3 Object Storage: Both the transformed application payloads from *AWS Glue* and the real-time stream deltas from *AWS Kinesis* land inside a private *AWS S3* bucket.

Multi-Tier Lake Isolation: *AWS S3* structurally isolates the incoming data by organizing it into distinct raw landing zones, historical archive zones, and highly optimized processed zones.

Immutable Persistence: The file write in *S3* locks the updated transaction state safely inside the *AWS storage* perimeter, acting as a highly durable, cost-effective data lake foundation for all corporate reporting.

Step 5: High-Performance Data Warehousing and Analytical Loading

AWS Redshift Integration: Modern data lake export pipelines automatically load the structured, processed tables from *AWS S3* directly into *AWS Redshift*.

Complex SQL Analytics: *AWS Redshift* stores the consolidated datasets in a high-performance, columnar data warehouse architecture, decoupling business intelligence querying from production systems.

Executive Dashboard Generation: Business intelligence tools execute complex analytical queries across *Redshift*, refreshing executive dashboards with up-to-date corporate metrics and multi-year historical trend tracking.

Step 6: Parallel Fabric Orchestration and Enterprise Governance

SAP Fabric Activation: While data lands in the cloud, instance-based specialized middleware layers like *SAP Datasphere* and *SAP Data Services* evaluate the active extraction streams.

Contextual Model Mapping: These specialized compute engines validate data lineage, apply enterprise-wide semantic business definitions, and map complex relationships between separate *SAP* systems (such as *S/4HANA*, *ECC*, and *BW*).

Downstream Table Provisioning: The synchronized enterprise data fabric builds refined operational views, programmatically provisioning structured analytical tables directly into target *AWS RDS* databases.

Step 7: Relational Microservice Consumption and Operational Staging

AWS RDS Processing: *AWS RDS* receives the structured relational updates, acting as a high-availability staging area and operational database for internal corporate microservices.

Microservice Execution: Custom cloud microservices query the *AWS RDS* tables to trigger automated supply chain workflows, material reorder processes, and inter-departmental finance updates.

Operational Synchronization: This ensures that internal corporate microservices, shipping applications, and logistical staging queues match active *SAP* source system states instantly without human intervention.

Step 8: Fault Isolation, Infrastructure Triage, and Operational Alerting

Boundary Error Interception: If a dedicated replication instance drops or a high-volume data payload fails schema validation, automated pipeline protection rules catch the failure instantly.

Infrastructure Triage: Specialized error-handler functions isolate the failed data batches, extracting the system stack traces and error codes to prevent data lake corruption or source system degradation.

AWS SNS Notification: The error-handling workflows send formatted exception alerts through *AWS Simple Notification Service (SNS)*. *SNS* dispatches critical infrastructure alerts straight to the *AWS Solution Support Team* for technical fixes, and forwards high-level operational trend updates directly to the *Business Users & Support Team* inbox.

Transaction Data-Flows

This section maps the structural journey into discrete, execution-based data flows. It separates real-time, synchronous transaction processing from non-blocking background logic and persistent analytical data mirroring.

1. Main Data Ingestion and Processing Flow

- **SAP Application Trigger:** Occurs when an operational delta, transaction update, or inventory adjustment is generated within the *SAP* application

layer. This is a non-blocking asynchronous push optimized to preserve core *ERP* system performance.

- **AWS AppFlow (Ingestion):** Operates in near-real-time as a fully managed integration service. It maps incoming *SAP* application structures to cloud-native data objects, handling initial schema translation and payload formatting automatically over encrypted channels.
- **AWS Glue (Integration & Cataloging):** Acts as the serverless data integration engine. It applies transformation rules to clean incoming *SAP* payloads, structures the tables into standard schemas, and registers the metadata instantly within the *AWS Glue Data Catalog*.
- **AWS S3 Landing Zone:** The foundational object storage data lake receives the records. Upon a successful commit, the data is partitioned into secure, immutable storage tiers (raw, archive, and processed) to maintain long-term historical consistency.

2. Real-Time Change Data Capture & Stream Processing Flow

- **SAP SLT Database Capture:** Simultaneously at the database tier, the *SAP Landscape Transformation Replication Server* intercepts live row-level modifications within the *SAP HANA* database. It streams these delta logs out using low-latency replication loops.
- **AWS Kinesis Data Streams:** Receives the live database mutation logs in real time. *Kinesis* processes the high-velocity ingestion records in micro-batches, maintaining a strict, time-ordered sequence of all inserts, updates, and deletes.
- **Data Lake Mirroring Loop:** Cloud-native pipeline consumer loops poll the *Kinesis* stream shards to extract the database deltas. These consumers serialize the micro-batches and dump them directly into the primary *AWS S3* storage lake to mirror live transactional states.

3. Analytics, Warehousing, and Fabric Orchestration Flow

- **AWS Redshift Loading:** Automated copy pipelines pick up the processed tables from the *AWS S3* landing zone. Data is loaded directly into a columnar data warehouse architecture to support intensive corporate business intelligence queries without impacting the production core.
- **Semantic Model Alignment:** Middleware systems like *SAP Datasphere* and *SAP Data Services* concurrently evaluate the ingestion paths. They map data lineage, enforce corporate metadata compliance, and reconcile

multi-table relationships across separate environments (such as *S/4HANA*, *ECC*, and *BW*).

- **AWS RDS Provisioning:** The synchronized enterprise data fabric structures refined analytical operational views. These validated views are pushed directly into target *AWS RDS* databases to feed downstream relational microservices, shipping applications, and transactional staging areas.

4. Error Handling & Fault Tolerance Flow

- **Pipeline Boundary Interception:** If a dedicated replication instance fails, an *AppFlow* connection times out, or a payload violates schema validation rules, the integration boundary raises an execution failure flag.
- **Infrastructure Triage Handling:** Specialized error-handling loops extract critical metadata from the failed execution trace, including the failure timestamp, specific error codes, and affected source records, to prevent data lake corruption.
- **AWS SNS Alerting:** The triage logic structures the exception payload into an actionable alert and sends it through *AWS Simple Notification Service*. *SNS* splits the alerts, sending technical infrastructure failures to the *AWS Solution Support Team* for engineering triage and broad trend summaries directly to the *Business Users & Support Team* inbox.

Security Architecture

This section outlines the defense-in-depth framework designed to protect enterprise data as it moves between the *Cloud analytics* platform and *SAP Source Systems*.

1. Ingestion and Perimeter Security

- **SAP Secure Communication & SNC**
Authenticates integration connections securely using *Secure Network Communications* (SNC) or *OAuth 2.0* protocols. This architecture eliminates the need to store long-lived *SAP* application usernames or cleartext passwords within the cloud analytics ecosystem.
- **AWS PrivateLink & Direct Connect**
Routes data directly over private network backbones using *AWS Direct Connect* or *AWS PrivateLink* instead of routing traffic across the public internet. This significantly reduces the attack surface and protects the enterprise payloads from external interceptors or *DDoS* attacks.

- **Gateway Access Control Lists (ACLs)**

Enforces strict perimeter-based firewall rules on specialized replication instances (like *SAP SLT* or *SAP Datasphere* hosts) to accept connection requests solely from designated source IPs. Any unauthorized external access attempts targeting the database or application layers are rejected immediately.

- **Ingestion IAM Roles**

Implements strict execution roles for *AWS Glue* and *AWS AppFlow* extraction pipelines using the principle of least privilege. Ingestion services are given explicit permissions solely for `s3:PutObject` and metadata cataloging on the designated landing zone paths.

2. Processing and Storage Security

- **KMS Managed Encryption**

Secures the core *AWS S3 Data Lake* and *AWS Redshift* data warehouse at rest using *AWS Key Management Service (KMS)* with *Customer Managed Keys (CMKs)*. This ensures complete governance over key rotation policies and provides a cryptographic audit trail.

- **Fine-Grained Stream IAM**

Restricts access to *AWS Kinesis Data Streams* by forcing stream consumer pipelines to assume dedicated *IAM* roles. These roles grant data read and processing rights strictly to specific *AWS Resource Names (ARNs)* belonging to the active shard paths.

- **VPC Endpoint Routing**

Deploys all data integration and transformation compute clusters inside *private subnets* of a *Virtual Private Cloud (VPC)* utilizing interface *VPC endpoints*. This configuration keeps all inter-service analytics and processing operations entirely isolated from external networks.

- **Secrets Manager Integration**

Uses *AWS Secrets Manager* to inject *SAP database* credentials, *RFC destination parameters*, or configuration values securely into runtime extraction environments. This workflow avoids the high-risk practice of hardcoding environment variables or plaintext passwords within the replication tools.

3. Analytics and Fabric Integration Security

- **S3 Block Public Access**

Enforces explicit "Block Public Access" configurations alongside strict bucket policies on all landing, raw, and processed *AWS S3* instances. This prevents any accidental exposure of the corporate transactional data or mirrored *SAP tables* to unauthorized public entities.

- **Row and Column-Level Security**

Limits analytical user access inside *AWS Redshift* and *AWS RDS* using fine-grained relational security configurations. User roles are restricted to view only data scopes relevant to their specific region or department, preventing unauthorized broad system access.

- **Secure Fabric Attestation**

Uses built-in security features within instance-based replication systems (*SAP Datasphere* and *Data Services*) to safeguard enterprise metadata models. Sensitive variables passed through the data fabric are encrypted in transit and mapped using locked system credentials.

- **Restricted SNS Publishing**

Applies strict resource policies to the *AWS SNS* notification topics to allow only authorized pipeline monitoring routines to call *SNS:Publish*. This structure blocks malicious actors from spoofing or injecting fake business notifications into the corporate email stream.

4. Governance and Error Security

- **CloudWatch PII Masking**

Implements data protection policies in *AWS CloudWatch* Logs to automatically detect and mask *Personally Identifiable Information* (PII) or sensitive financial columns. This prevents proprietary *SAP* corporate fields from appearing in plain text across application logs.

- **CloudTrail Control Logging**

Monitors and logs every *API transaction* across all *cloud analytics* components using *AWS CloudTrail*. This provides the compliance team with immutable, time-stamped logs of exactly who accessed or modified the data platform infrastructure.

- **Encrypted Dead-Letter Queues**

Routes any failed *AppFlow* extractions or un-routable data pipeline payloads directly into an encrypted *AWS SQS* Dead-Letter Queue (*DLQ*) or error-handling storage bucket. This mechanism guarantees that broken transactions are isolated safely for debugging without causing data lake corruption.

- **SNS Server-Side Encryption**

Activates *Server-side encryption* (SSE) on the administrative *SNS* alert topics using *AWS KMS* keys. This provides a baseline layer of data protection for internal system error payloads before they are delivered to the operational support team.

Disaster Recovery (DR) Plan

This plan ensures business continuity, system availability, and zero data corruption if an infrastructure failure or natural disaster takes down a cloud availability zone or the on-premises data centre.

1. Multi-Region Replication and Infrastructure Strategy

- **S3 Cross-Region Replication (CRR)**

Automates the asynchronous replication of raw, landing, and processed *SAP* data files from the primary *AWS S3* bucket to a target bucket in the disaster recovery region. *S3 Versioning* is enabled on both buckets to prevent data loss from accidental deletion, overwrites, or file corruption.

- **Redshift Cross-Region Snapshot Copy**

Enables fully managed cross-region snapshot replication for *AWS Redshift* to copy automated and manual data warehouse snapshots to the secondary *AWS region*. This configuration ensures that if the primary region goes offline, an identical copy of the analytical data store is instantly available for disaster recovery instantiation.

- **Infrastructure as Code (IaC) Parity**

Maintains synchronized deployments of all *AWS Glue* jobs, *AWS Kinesis streams*, *AWS RDS* instances, and *AWS SNS* topics across both regions using *Terraform* or *AWS CloudFormation*. This guarantees configuration consistency and eliminates drift, ensuring the secondary region is always ready to accept live traffic.

- **Pre-Configured Ingestion Standby Bundles**

Deploys inactive, identical instances of *AWS AppFlow* connections and replication instance scripts inside the secondary region. These connections point to the same *SAP* environments but remain dormant until a failover event occurs, avoiding duplicate processing, data collision, or source system overhead issues.

2. Monitoring, Health Checks, and Failover Automation

- **Route 53 Application Recovery Controller (ARC)**

Monitors the end-to-end health of the primary data integration pipelines using *synthetic canaries* and regional *CloudWatch* metrics. *ARC* manages routing control states, enabling automated or single-click routing redirection during high-severity service disruptions or replication instance failures.

- **Kinesis Replication Failover Mechanics**

Utilizes backup ingestion pathways to automatically redirect incoming real-time *Change data capture* (CDC) streams if the primary *Kinesis data stream* becomes unreachable. This architecture safely buffers incoming payloads at the collection edge while transitioning analytical processing workloads to the backup region.

- **CloudWatch Cross-Region Alarms**

Aggregates error metrics, *AWS Glue* job failures, and *AWS AppFlow* extraction timeouts into a centralized operational dashboard. When specified threshold limits are breached consecutively for 3 minutes, it triggers an automated failover sequence via *AWS Systems Manager*.

- **Automated Step Functions Runbook**

Executes a coordinated failover orchestration script via *AWS Step Functions* when an outage is declared. This runbook activates the secondary *AppFlow* extraction paths, updates *DNS routing* records for the replication tools, and flips the execution state of the backup data processing engines.

3. SAP-Side Rerouting and Data Recovery

- **SAP RFC Destination Updates**

Modifies the endpoint parameters within *SAP Remote Function Call* (RFC) destinations dynamically using automated scripts or landscape management tools during a disaster scenario. This redirects the outbound *SAP application* payload stream away from the failed primary region to the healthy secondary *AWS endpoint*.

- **Replication Pointer Utilization**

Leverages *SAP SLT*'s transaction logging retention window by utilizing un-replicated delta pointers if the cloud ingestion boundary suffers gaps during the failover window. The secondary ingestion instance requests

past change logs starting from the exact last-known successful transaction sequence number to achieve zero data loss.

- **Bulk Loading Failover Strategy**

Switches the scheduled data lake synchronization loops to use high-throughput bulk export methods when recovering from long outages. This optimizes processing efficiency and handles large backlogs of staged *SAP source* data without hitting cloud extraction limits or choking network bandwidth.

- **Bidirectional Correlation Tracking**

Embeds a unique, immutable *UUID* tracking ID within the *SAP Source System* payloads that stays attached to the data across *AWS Glue*, *S3*, and *Redshift*. This shared reference allows automated reconciliation scripts to quickly identify dropped messages or missing tables between the two platforms during a regional flip.

4. Failback, Data Reconciliation, and Testing

- **Regional Parity and Delta Validation**

Verifies that the primary *region* is fully stable and that all data deltas have completed replication before initiating a failback. The operations team reviews data lake file counts, *S3* replication lag, and *Redshift* snapshot metrics to confirm that both regional databases are in absolute sync.

- **Traffic Draining Phase**

Allows active data blocks inside the secondary region's *Kinesis streams*, staging buckets, and *AWS RDS* queues to process completely before shifting traffic back. This controlled cooldown period ensures that no mid-flight operations are cut off or corrupted during the transition.

- **Phased Traffic Shifting**

Employs an incremental canary routing approach using *Route 53* to shift 10% of the data ingestion traffic back to the primary region first. System performance and ingestion error rates are monitored for a 15-minute soak period before routing the remaining 90% of production traffic.

- **Chaos Engineering and Game Days**

Mandates quarterly, non-disruptive disaster recovery testing using *AWS Fault Injection Service* (FIS) to simulate regional outages and replication failures in a staging environment. These simulations validate that automated recovery patterns perform successfully within the targeted 15-minute recovery window.

Architecture Tradeoffs

1. Ingestion Latency vs. Operational Overhead

- **The Benefit:** Leveraging Path A (*AWS AppFlow* and *AWS Glue*) provides a completely serverless, low-code ingestion pipeline. This eliminates the need to patch, provision, or manage underlying virtual machine servers, significantly lowering day-to-day administrative overhead.
- **The Trade-off:** Serverless connectors operate on scheduled batch intervals or micro-batches, introducing data latency. This path is incapable of delivering true, sub-second real-time streaming required for live operational event monitoring.
- **The Mitigation:** Implement a multi-path model that routes low-latency analytical requirements through Path B (*SAP SLT* with *AWS Kinesis*). Use Path A exclusively for high-volume, non-time-sensitive master data tables (e.g., nightly product catalog updates).

2. High-Frequency Real-Time Replication vs. SAP Source System Performance

- **The Benefit:** Deploying Path B using *SAP SLT* allows for continuous *Change Data Capture* (CDC) at the database layer. This ensures that the *AWS Analytics Landing Zone* mirrors live *transactional ERP* events with near-zero latency.
- **The Trade-off:** Continuous database log reading and data streaming consume significant CPU and memory resources on the source infrastructure. If unconstrained, this replication activity can degrade performance for concurrent *OLTP* production users.
- **The Mitigation:** Configure *SAP SLT* on a dedicated, isolated compute instance outside of the primary production *ERP cluster*. Set strict resource throttling limits within *SAP*, and isolate extraction schedules to replicate only highly critical transactional tables in real time.

3. Deep Decoupled Storage vs. Analytical Query Latency

- **The Benefit:** Organizing data into a multi-tier *AWS S3 Data Lake* establishes an immutable, highly durable, and cost-effective foundation. This decouples the transactional source systems entirely from downstream analytical consumers, preventing performance impact.

- **The Trade-off:** Querying raw object storage directly (via tools like *AWS Athena*) introduces latency penalties for complex analytical operations. It does not natively deliver the single-digit millisecond response times required for executive *BI dashboards*.
- **The Mitigation:** Orchestrate an automated downstream loading pipeline that structures and copies processed *S3* data into *AWS Redshift*. *Redshift* uses a high-performance, columnar architecture to deliver low-latency execution for heavy enterprise *SQL reporting*.

4. Custom Application Add-ons vs. Architecture Portability

- **The Benefit:** Utilizing Path C with embedded partner *ABAP add-ons* allows the system to push pre-aggregated, application-level data payloads directly. This layer understands complex *SAP* internal logic natively, reducing data transformation work inside *AWS*.
- **The Trade-off:** Relying on embedded *ABAP* code introduces significant proprietary lock-in and increases technical debt. Upgrading the core *SAP ERP* version or migrating to a different cloud vendor requires expensive recoding and testing cycles.
- **The Mitigation:** Enforce a strict "Clean Core" strategy by standardizing the embedded *ABAP* code to output only generic, universally accepted *JSON* payloads. Shift all complex business transformation and formatting logic out of *SAP* and into scalable, *cloud-native AWS Glue* jobs.

5. Multi-Table Semantic Alignment vs. Storage Cost Optimization

- **The Benefit:** Deploying instance-based specialized middleware layers like *SAP Datasphere* and *SAP Data Services* allows the architecture to map complex metadata models, preserve relational lineage, and enforce corporate data governance across separate *SAP systems* before the data reaches long-term storage.
- **The Trade-off:** Maintaining continuous running compute instances to manage these large-scale *data fabrics* and provisioning redundant staging views across target *AWS RDS* databases drastically increases monthly infrastructure costs and introduces structural storage duplication.
- **The Mitigation:** Implement aggressive *AWS S3 lifecycle* retention tiers that automatically transition cold, un-orchestrated raw landing data to *S3 Glacier Deep Archive* within 14 days, and use *AWS Redshift Spectrum* to

query historical data lake tables directly without duplicating them in relational storage tiers.

Architecture Decision Records

1. Selection of AWS Data Ingestion Pattern

- **Decision**

Implement a hybrid ingestion strategy utilizing both *AWS Managed Services* (Path A) and Instance-Based/Partner Solutions (Path B) to ingest data from *SAP Source Systems* into the *AWS Analytics Services Landing Zone*.

- **Rationale**

AWS AppFlow and *AWS Glue* reduce operational overhead for standard *SAP HANA database* extractions and SaaS integrations. Specific business requirements demand specialized replication pipelines like *SAP SLT* (Landscape Transformation), *SAP Datasphere*, or validated *Partner Solutions* to capture delta changes and complex application logic efficiently. ports, while a *VPN Gateway* provides a cost-effective, automated disaster recovery path.

2: Integration of SAP Application Layer Extractors

- **Decision**

Utilize custom ABAP add-ons and partner solutions embedded directly within the SAP ABAP stack (Path C) for application-level data extraction.

- **Rationale**

Extracting directly from the *ABAP* layer (*S/4HANA*, *ECC*, *BW*) preserves complex business logic, security configurations, and standard *SAP extractors*. This avoids the complexity of rebuilding deep application-layer logic inside AWS.

3. Establishment of a Central Analytics Landing Zone

- **Decision**

Deploy a unified *AWS Analytics Services Landing Zone* containing *AWS S3*, *AWS Redshift*, *AWS Kinesis*, and *AWS RDS* to serve as the destination for all *SAP* data.

- **Rationale**

AWS Redshift provides high-performance data warehousing for relational business intelligence. *AWS Kinesis* enables real-time stream processing for

operational data. *AWS RDS* supports specialized operational databases or downstream application storage.

4. Support for Multi-Platform SAP Hosting

- **Decision**

Design ingestion patterns to support *SAP* source systems regardless of whether they are hosted on *AWS* natively, via *RISE* with *SAP*, or remaining On-Premises.

- **Rationale**

This ensures future-proof connectivity during multi-phase cloud migrations. It minimizes architectural changes to *AWS* ingestion pipelines when the underlying *SAP* hosting model changes.

5. Standardizing Replication for Real-Time SAP Delta Data

- **Decision**

Deploy *SAP Landscape Transformation (SLT)* within the instance-based solutions block (Path B3) to feed the *AWS Analytics Landing Zone*.

- **Rationale**

SAP SLT provides trigger-based, real-time data replication from *SAP ECC* or *S/4HANA* systems. This minimizes performance impacts on the source system while capturing database-level delta changes (*CDC*) immediately.

6 Standardizing SaaS and API-Driven Ingestion

- **Decision**

Use *AWS AppFlow* as the primary fully managed service for API-driven and SaaS integration patterns from *SAP HANA* Cloud environments.

- **Rationale**

AppFlow provides a no-code, secure, and fully managed connector infrastructure. It eliminates the need to provision, manage, or scale integration servers, reducing total cost of ownership (*TCO*).

7. Decoupling Storage and Compute via Data Lake Pattern

- **Decision**

Designate *AWS S3* as the initial landing layer and foundational object store within the *AWS Analytics Services Landing Zone*.

- **Rationale**

S3 serves as an immutable landing zone for raw data files before processing. This decouples ingestion compute (*AppFlow*, *Glue*, *SLT*) from long-term analytical compute (*Redshift*), optimizing costs and system reliability.

8. Implementing Purpose-Built Storage for Streaming Data

- **Decision**
Route high-frequency, event-driven, or streaming data from *SAP* source systems into *AWS Kinesis*.
- **Rationale**
Traditional batch tools or relational databases cannot handle high-throughput, sub-second telemetry or event data efficiently. *AWS Kinesis* captures and buffers these real-time data streams reliably before persisting them into *AWS S3* or *Redshift*.

9. Provisioning Managed ET/ETL for Database-Level Access

- **Decision**
Utilize *AWS Glue* as the serverless data integration service for direct extraction from the *SAP HANA* database layer.
- **Rationale**
AWS Glue natively scales to discover, prepare, and transform structured data from databases. Its serverless nature ensures that compute power is only consumed and paid for during active *ETL* run windows.

10. Multi-Engine Target Database Provisioning

- **Decision**
Provision both *AWS Redshift* (Columnar OLAP) and *AWS RDS* (Row-oriented *OLTP*) within the destination landing zone.
- **Rationale**
AWS Redshift handles heavy aggregation, massive historical datasets, and complex business intelligence reporting queries. *AWS RDS* provides predictable, low-latency performance for transactional operational data stores (*ODS*) and smaller application metadata storage.

AWS Well-Architected Framework Review

An evaluation of this architecture against the design principles of the AWS Well-Architected Framework reveals the following target alignments and gaps:

1. Security Pillar

- **Strengths**

Separating managed services (*AppFlow*), instance-based tools (*SLT*), and *ABAP* add-ons creates isolated, purpose-built data perimeters. Extracting data via the *ABAP* layer (Path C) allows you to respect and preserve *SAP*'s application-level security, roles, and authorizations.

- **Recommendations:**

Ensure all traffic from On-Premises or *RISE* with *SAP* environments travels over a dedicated *AWS Direct Connect* or an encrypted *AWS Site-to-Site VPN*. Restrict *AWS AppFlow*, *AWS Glue*, and *SAP SLT* execution roles to write only to specific, isolated prefixes inside the *AWS S3 landing zone*.

2. Reliability Pillar

- **Strengths:**

Using *AWS S3* as the core landing zone ensures that downstream analytics processing failure will not disrupt the upstream *SAP* source systems. Incorporating *AWS Kinesis* provides a resilient buffer to absorb unexpected spikes in streaming data velocity without dropping events.

- **Recommendations:**

Design *cross-region replication* (CRR) for critical *S3* landing buckets and maintain *multi-AZ* deployments for *AWS Redshift* and *AWS RDS*. Configure *AWS AppFlow* and *AWS Glue* jobs with robust retry logic and *Dead Letter Queues* (DLQ) to handle transient network hiccups between *SAP* and *AWS*.

3. Cost Optimization Pillar

- **Strengths:**

The heavy usage of *AWS AppFlow*, *AWS Glue*, and *AWS S3* ensures you only pay for the exact compute used during data extraction and processing. The architecture naturally allows for data lifecycle policies, moving older data from expensive relational databases to low-cost *S3* tiering.

- **Recommendations:**

Monitor and rightsize the *EC2/Compute* instances hosting the "Instance-Based Solutions" (Path B) like *SAP SLT* or *Partner Solutions*, as these run 24/7. Transition raw, unprocessed *SAP source* files in *S3* to *Glacier* Instant Retrieval or Flexible Retrieval after a short retention window (e.g., 7 to 14 days).

4. Performance Efficiency Pillar

- **Strengths:**

Splitting workloads between *AWS Redshift* (for massive analytical queries) and *AWS RDS* (for transactional workloads) prevents resource contention. *AWS Kinesis* offloads real-time processing from standard batch systems, ensuring low-latency data availability.

- **Recommendations:** Configure *AWS Glue* and *Partner Solutions* to convert raw *SAP* data into columnar, compressed formats like *Apache Parquet* or *ORC* before writing to *S3*. Maximize the use of *Change Data Capture (CDC)* via *SAP SLT* to ingest only modified records, preventing massive network overhead and performance degradation on the *SAP database*.

5. Operational Excellence Pillar

- **Strengths:**

The clear demarcation of ingestion pathways simplifies troubleshooting and establishes clear team ownership boundaries. Leveraging *AWS Glue* and *AppFlow* minimizes the operational burden of patching and maintaining underlying OS and integration servers.

- **Recommendations:**

Funnel logs from *AWS Glue*, *AppFlow*, *Kinesis*, and *SAP* application logs into *AWS CloudWatch* or an *enterprise SIEM*. Set up alerts for ingestion pipeline failures. Deploy *Infrastructure as Code (IaC)*: Codify the entire *AWS Analytics Landing Zone* and integration pipelines using *AWS CDK* or *Terraform* to guarantee identical Dev, QA, and Production environments.

6. Sustainability Pillar

- **Strengths:**

Using serverless components (*S3*, *AppFlow*, *Glue*) maximizes hardware utilization on *AWS* infrastructure, minimizing idle energy consumption.

- **Recommendations:**

Avoid keeping identical copies of raw data across *S3*, *Redshift*, and *RDS*. Use features like *Redshift Spectrum* to query data directly in *S3* where appropriate. Align heavy, non-real-time *AWS Glue* or Partner extraction jobs to run during off-peak hours to flatten energy consumption curves.

Architecture Risks

1. Monolithic Gateway Bottleneck

- **The Flaw:**
Routing all instance-based replication, *ABAP extractors* through a single, unscaled replication server or shared network appliance.
- **The Risk:**
High-volume parallel extractions during peak *SAP* business hours will exhaust *CPU/memory* on the replication host, dropping data packets and stalling downstream *AWS analytics dashboards*.
- **The Fix:**
Deploy the instance-based solutions across an *Auto Scaling Group* (ASG) of *EC2* instances behind a *Network Load Balancer* (NLB) to dynamically distribute processing loads.

2. Delta Sync Drift and Split Brain

- **The Flaw**
Relying heavily on time-stamp-based or batch extraction queries rather than database-level *Change Data Capture* (CDC) for high-frequency tables.
- **The Risk**
If an ingestion job fails mid-run, deleted or rapidly updated *SAP* records will be missed, creating severe data discrepancies between *SAP* and *AWS Redshift*.
- **The Fix**
Enforce trigger-based *CDC* via *SAP SLT* or validated Partner Solutions to stream transaction logs into *AWS Kinesis*, ensuring absolute transactional consistency.

3. Network Saturation Over Shared Pipes

- **The Flaw**
Mixing heavy, raw *SAP* data extraction traffic with standard corporate or production application traffic over the same internet or *VPN connection*.
- **The Risk**

Large historical database extractions will saturate network bandwidth, causing severe latency spikes for operational *SAP* users and critical business workflows.

- **The Fix**

Provision a dedicated *AWS Direct Connect* with a separate *Transit Virtual Interface* (VIF) explicitly allocated for analytics ingestion data traffic.

4. Data Lake Sprawl and Performance Degradation

- **The Flaw**

Storing raw *SAP* data directly in *AWS S3* as uncompressed, unpartitioned, millions of small *JSON* or *CSV* files.

- **The Risk**

AWS Glue, *Athena*, or *Redshift Spectrum* queries will slow down dramatically and incur massive scan costs because they must open millions of tiny files on every run.

- **The Fix**

Implement an *AWS Glue ETL* job that immediately batches, partitions by date, and converts incoming raw data into a columnar format like *Apache Parquet*.

5. Over-Privileged Security Context

- **The Flaw**

Using a single *SAP* service account with sweeping administrative authorizations (*SAP_ALL*) to facilitate the extraction connectors.

- **The Risk**

A security compromise of the *AWS integration credentials* could allow an attacker to gain full read/write access to the entire core *SAP ERP database*.

- **The Fix**

Enforce strict least-privilege access by creating dedicated *SAP* communications users restricted solely to the specific *RFCs*, *BAPIs*, or *tables* required for extraction.

6. API Rate-Limiting and Throttling

- **The Flaw**

Using *standard OData* or *REST APIs* (Path A) for massive volume bulk-data extraction out of *SAP HANA Cloud* or *SaaS* layers.

- **The Risk**

The *SAP application layer* will trigger rate-limiting or timeout errors, killing long-running *AWS AppFlow* or *Glue* extraction jobs mid-transit.

- **The Fix**

Restructure ingestion logic to use paginated *API* calls for smaller operational deltas, and pivot to native database-level or bulk file-export mechanisms for historical loads.

7. SAP System Upgrades Breaking Extractor Schemas

- **The Flaw**

Relying on direct database table extractions (*B/HANA* layer) without an abstraction layer during major *SAP kernel* or *S/4HANA* version upgrades.

- **The Risk**

Schema changes, renamed fields, or dropped tables during the *SAP* upgrade will instantly break downstream *AWS Glue* and *Redshift* pipelines, halting corporate reporting.

- **The Fix**

Standardize on *SAP CDS (Core Data Services) Views* or *standard OData APIs* as the extraction interface, isolating *AWS* from underlying physical database modifications.

8. Unbounded Operational Database Growth

- **The Flaw**

Landing all historical *transactional SAP* data directly into *AWS RDS* without a data aging or archiving policy.

- **The Risk**

AWS RDS storage and *IOPS* costs will scale exponentially, and query performance for downstream operational applications will severely degrade over time`

- **The Fix**

Enforce a strict retention window on *AWS RDS*, configuring automated pipelines to offload older cold data into *AWS S3* utilizing *Redshift Spectrum* for historical queries.

9. Lack of End-to-End Lineage Tracking

- **The Flaw**

Ingesting *SAP* data through multiple disparate pathways into the *AWS Landing Zone* without a unified metadata catalog.

- **The Risk**
Data consumers lose visibility into data origins, making it impossible to audit calculations, verify data compliance, or troubleshoot upstream calculation errors.
- **The Fix**
Implement a centralized governance layer using *AWS Glue Data Catalog* and *AWS DataZone* to automatically crawl, tag, and trace data ancestry across all ingestion pathways.

10. Clock Desynchronization Across Hybrid Environments

- **The Flaw**
Operating the *SAP source* environments (*On-Premises/RISE*) and the *AWS Analytics Zone* on separate, unaligned time-keeping servers.
- **The Risk**
Time-stamp-based delta tracking, time-sensitive security tokens, and data sequencing metrics will mismatch, leading to out-of-order data processing and authentication failures.
- **The Fix**
Mandate that all *SAP* hosts and *AWS infrastructure* sync to a unified, highly accurate time source using *AWS Time Sync Service* or a shared enterprise *Network Time Protocol* (NTP).

Architectural Recommendations

1 . Implement Columnar Storage

- **Action**
Configure your *AWS Glue ETL* pipelines to convert raw *SAP* data (*CSV/JSON*) into *Apache Parquet* format before saving it to *AWS S3*.
- **Recommendation**
This reduces *S3* storage costs, minimizes network data transfer, and increases downstream query speeds in *AWS Redshift* by up to 10x.

2. Enforce a Hub-and-Spoke Network Topology

- **Action**
Connect your On-Premises or *RISE* with *SAP* environments to *AWS* using *AWS Transit Gateway* connected to an *AWS Direct Connect gateway*.
- **Recommendation**
This centralizes traffic routing, simplifies network management across

multi-account structures, and ensures secure, dedicated bandwidth for heavy *SAP data extractions*.

3. Utilize SAP CDS Views for Core Extractions

- **Action**

Mandate that all application-layer data extractions utilize *SAP Core Data Services* (CDS) Views rather than direct database table access.

- **Recommendation**

This protects your *AWS* pipelines from breaking during future *SAP* system upgrades, preserves *SAP*'s internal business logic, and respects existing application-level authorizations.

4. Deploy Automated S3 Lifecycle Policies

- **Action**

Apply an *S3 Lifecycle* rule to transition raw *SAP landing* files to *AWS S3 Glacier* Instant Retrieval after 7 days, and permanently delete or archive them after 30 days.

- **Recommendation**

This minimizes high-tier object storage costs by ensuring you only store active, processed data in your hot tier, while keeping raw historical files in ultra-low-cost storage.

5. Establish a Single Source of Truth Metadata Catalog

- **Action**

Deploy the *AWS Glue Data Catalog* to crawl all incoming *SAP* files in the *AWS S3 landing zone* automatically.

- **Recommendation**

This creates a central data dictionary that enables schema evolution tracking, simplifies data governance, and lets users seamlessly query data across *S3* and *Redshift*.

6. Isolate Environments via Multi-Account Strategy

- **Action**

Segregate your integration pipelines into separate *AWS accounts* for Development, Testing, and Production using *AWS Organizations*.

- **Recommendation**

This prevents testing workloads from impacting production system performance and limits the blast radius of any potential configuration errors or security breaches.

7. Harden Security via Least-Privilege IAM Roles

- **Action**
Create dedicated, restricted *AWS IAM* execution roles for *AWS AppFlow* and *AWS Glue*, granting permission *only* to their specific *S3* buckets.
- **Recommendation**
This eliminates the risk of broad lateral movement within your *AWS* cloud environment if an integration connector credential is compromised.

8. Implement Real-Time Dead Letter Queues (DLQ)

- **Action**
Configure *AWS AppFlow* pipelines and *AWS Glue* jobs to redirect any malformed *SAP* records or failed payloads to an isolated *S3 Dead Letter Queue* bucket.
- **Recommendation**
This ensures that a single corrupt record or network glitch won't halt the entire data pipeline, allowing clean data to keep moving while you troubleshoot errors offline.

9. Leverage Redshift Spectrum for Cold Data

- **Action**
Move old, historical *SAP* data out of *AWS Redshift* local storage and store it in *AWS S3*, using *Redshift Spectrum* to query it externally when needed.
- **Recommendation**
This stops your *AWS Redshift* cluster storage costs from growing exponentially while maintaining full query capability over your entire historical *data lake*.

10. Centralize Monitoring with CloudWatch Dashboards

- **Action**
Stream logs from *AWS AppFlow*, *AWS Glue*, *AWS Kinesis*, and your *SAP replication* servers into a single *AWS CloudWatch* dashboard.
- **Recommendation**
This gives your operations team real-time visibility into pipeline health, data velocity, and error rates, drastically reducing the time it takes to detect and fix ingestion failures.

Boundary Conditions for Reference Architecture

1 Maximum Single-Table Payload Size

- **The Issue**

Ingesting massive *SAP* cluster or database tables (e.g., *ACDOCA* or *BSEG* containing billions of rows) via standard *API* or managed application layers.

- **The Condition**

Individual table extractions exceeding 1 Terabyte (TB) in size or 50 million rows per batch must bypass *AWS AppFlow* and *standard OData APIs*, routing instead through *SAP SLT* utilizing parallelized database-level slicing.

2. Pipeline Latency and Velocity Thresholds

- **The Issue**

Choosing between *batch*, *near-real-time*, and sub-second streaming pipelines for different business use cases.

- **The Condition**

Data requiring an end-to-end latency of under 5 seconds must utilize *AWS Kinesis* (Streaming); data with a 5-minute to 24-hour tolerance must use *AWS Glue* or *AppFlow* batch intervals; sub-second hard real-time operational loops must remain entirely inside the native *SAP system*.

3. Network Bandwidth Allocation

- **The Issue**

Large data extractions overwhelming the hybrid network connectivity shared with production *ERP* traffic.

- **The Condition**

This architecture requires a minimum dedicated network throughput of 1 Gbps via *AWS Direct Connect*, if data volume peaks exceed 70% of available bandwidth during business hours, extractions must be rate-limited or scheduled strictly during designated off-peak windows.

4. SAP Source Compatibility and Application Versioning

- **The Issue**

Handling legacy *SAP* systems that lack modern *API* frameworks or cloud-native database architectures.

- **The Condition**

Source systems must be on *SAP NetWeaver 7.4* or higher to utilize *AWS AppFlow* or native *ABAP* extractors any legacy systems below this level are restricted to file-based landing (*CSV* via secure *FTP*) or third-party database replication agents.

5. Target Warehouse Storage Limits

- **The Issue**

Managing the physical storage capacity and cost scaling of the analytical target database..

- **The Condition**

Operational data tracking inside *AWS RDS* is bounded to a maximum of 64 Terabytes (TB) per instance; any datasets projected to grow beyond this limit must be architected into *AWS Redshift* or stored as cold *Parquet* files in *AWS S3*.

Appendix A: Non-Functional Requirements (NFR) Matrix

This matrix defines the critical quality attributes, operational constraints, and performance benchmarks that the system must satisfy to ensure long-term stability, security, and scalability.

NFR Category	Target Objective	Engineering Validation Mechanism(Tools)	Architecture Component Mapping
Availability	99.9% uptime for data pipelines 99.99% uptime for analytics layer	- AWS Resilience Hub - AWS CloudWatch Synthetics	AWS S3 AWS RedshiftAWS RDS (Multi-AZ)
Performance	- End-to-end data latency < 15 mins for batch - Stream latency < 10 seconds	- AWS CloudTrail timestamp analysis - AWS Kinesis Data Streams metrics	AWS AppFlow AWS Glue AWS Kinesis
Scalability	- Auto-scale ingestion compute up to 10x baseline - Handle 2TB+ daily data volume	- AWS Glue Auto Scaling logs - AWS Distributed Load Testing Solution	AWS Glue Serverless AWS Redshift Serverless
Security	100% data-at-rest & in-transit encryption - Zero public IP exposure	- AWS Security Hub - AWS IAM Access Analyzer - AWS GuardDuty	AWS KMS AWS PrivateLink AWS Transit Gateway
Data Retention	- Raw data stored for 7 years minimum - Operational data kept 90 days in database	- AWS S3 Lifecycle Policy compliance audits - AWS Backup reports	AWS S3 Glacier AWS RDS Storage
Recoverability	- Recovery Point Objective (RPO) < 15 mins - Recovery Time Objective (RTO) < 4 hoursRecovery Point Objective (RPO) < 1 min	- AWS Backup automated restore testing - Chaos engineering via AWS Fault Injection Service.	AWS RDS Backups AWS Redshift Snapshots
Observability	100% pipeline failure alerts sent within 2 mins	AWS CloudWatch Alarms & SNS	AWS CloudWatch AWS DataZone

	Complete lineage tracking	AWS Glue Data Catalog auditing	
Sustainability	- Reduce carbon footprint by minimizing idle compute - Maximize hardware utilization targets	- AWS Customer Carbon Footprint Tool - AWS Compute Optimizer	AWS Glue Serverless AWS AppFlow S3 Lifecycle Tiers
Data Governance	100% of tables tagged with data classification - Automatic schema drift detection	- AWS DataZone dashboard metrics - AWS Glue Crawler logs	AWS Glue Data Catalog AWS DataZone
Interoperability	- Standard open data formats for downstream usage - Multi-BI tool access concurrency	- Apache Parquet compliance checking tools - AWS Redshift concurrency scaling logs	AWS S3 (Parquet) AWS Redshift
Maintainability	100% infrastructure deployed as code (IaC) - Pipeline deployment drift < 10 mins	- AWS CloudFormation Drift Detection - Terraform plan validation pipelines	AWS CDK / Terraform AWS Systems Manager
Financial Governance	- Alerting triggers at 10% budget overruns - Cost allocation attribution to 100% of resources	- AWS Budgets alerting engine - AWS Cost Anomaly Detection	AWS Cost Categories Resource Tagging Framework
Data Quality	- Automated validation rules on incoming data records - Quarantine 100% of corrupt payloads	- AWS Glue Data Quality metrics - S3 Dead Letter Queue (DLQ) alerts	AWS Glue Data Quality Engine S3 Quarantine Buckets

Appendix B: Cost Optimization & Attribution Matrix

This appendix provides a structured evaluation framework for linking operational expenses to their true business drivers, ensuring financial accountability and resource efficiency.

Cost Domain	Identified Bleed Point	Architectural Fix / Optimization Strategy	Expected Financial Impact
Compute	 OpEx	Implement an AWS Auto Scaling group (ASG) triggered by CPU utilization and scheduled scaling for off-peak windows.	30% to 50% reduction in replication instance costs.
Storage	 OpEx	Enforce S3 Lifecycle policies to move raw files to S3 Glacier Instant Retrieval after 7 days and Deep Archive after 30 days.	up to 70% reduction in data lake storage footprint costs.
Database	 OpEx	Offload older transactional data to AWS S3 as Parquet files and query it dynamically using Redshift Spectrum.	40% reduction in data warehouse compute and local storage provisioning costs.
Network	 OpEx	Deploy VPC Endpoints (AWS PrivateLink) to route ingestion traffic privately over the internal AWS network backbone.	20% to 35% savings on monthly data egress and transfer line charges.
Data Licensing	 CapEx	Transition extraction pipelines to serverless, consumption-based options like AWS Glue and AWS AppFlow.	Eliminates upfront software licensing overhead, switching costs to true pay-as-you-go utility models.
ETL Processing	 OpEx	Refactor AWS Glue ETL logic to process data using daily S3 folder partitioning and native SAP Change Data Capture (CDC).	50% drop in serverless Data Processing Unit (DPU) runtime costs.
Database Compute	 OpEx	Migrate fixed-size instances to AWS RDS Aurora Serverless v2 to automatically scale capacity up and down based on active query demand.	40% to 60% reduction in database instance runtime expenses.

Hardware	 CapEx	Decommission local physical gateway hardware and route connections directly to AWS using virtualized AWS Transit Gateway setups.	Eliminates immediate capital asset acquisition and refresh costs.
Monitoring	 OpEx	Refactor log metric filters to group events, and lower retention periods for raw, non-critical pipeline debugging logs to 14 days.	15% to 25% reduction in monthly cloud monitoring and logging bills.

Appendix C: Enterprise Security Risk Register

This register serves as the central repository for identifying, assessing, and tracking security vulnerabilities and threats across the organization, enabling leadership to make informed, risk-based decisions.

Security Vulnerability Description	Likelihood (1-5)	Impact (1-5)	Risk Rating (1-25)	Core Technical Mitigation Strategy
Credential Exposure / Hardcoded API Keys: Compromise of access keys used by AWS AppFlow or custom ABAP connectors to authenticate between SAP and AWS.	3	5	15 (Medium)	Mandate the use of AWS Secrets Manager with automatic 30-day credential rotation and IAM policy enforcement.
Lateral Movement via Over-Privileged Roles: An attacker compromises an AWS Glue or EC2 instance and uses sweeping IAM roles (*actions) to access corporate networks.	2	5	10 (Medium)	Enforce strict Least-Privilege IAM Policies and use AWS IAM Access Analyzer to eliminate wildcard resource permissions.
Man-in-the-Middle (MitM) Interception: Eavesdropping or altering of sensitive financial/HR SAP data in-transit across public internet endpoints during extraction	2	5	10 (Medium)	Enforce TLS 1.3 encryption for all connections and route traffic through AWS PrivateLink and encrypted Site-to-Site VPN or Direct Connect.
Unprotected S3 Storage (Data Leak): Misconfiguration of AWS S3 landing zone buckets leading to public exposure of raw, unencrypted SAP data extracts.	2	5	10 (Medium)	Enable S3 Block Public Access at the account level, enforce AWS KMS customer-managed keys (CMK), and deploy AWS SCPs to block unencrypted uploads.
Denial of Service (DoS) on SAP Host: Unthrottled, concurrent AWS Glue or AppFlow API requests overwhelming the SAP	3	3	9 (Medium)	Implement API Gateway rate-limiting policies and configure SAP-side gateway profile parameters to restrict

Application Layer, knocking core business operations offline.				maximum concurrent RFC/OData connections.
Data Ingestion Spill / Classification Drift: Highly confidential SAP data (e.g., PII, PCI, or payroll data) leaking into unapproved low-security analytics environments or public folders.	3	3	9 (Medium)	Deploy AWS Macie to automatically scan the S3 landing zone for sensitive data patterns, and enforce tagging-based attribute access control (ABAC).
Unauthorized SAP Database Access: Use of generic database service accounts (like SAP_ALL) by integration tools, bypassing application-level access controls and change logging.	2	4	8 (Low)	Enforce dedicated SAP Communication Users restricted to specific CDS views or RFC function modules, disabling direct table-level database access.
Ransomware / Malicious Data Deletion: Compromised admin or pipeline identity maliciously deleting or encrypting the SAP data lake history stored in AWS S3	2	5	10 (Medium)	Enable S3 Versioning and enforce S3 Object Lock in Compliance mode with Multi-Factor Authentication (MFA) Delete to ensure data immutability.
Orphaned Access and Inactive Accounts: Former employees or stale service accounts retaining legacy access tokens or SSH keys to instance-based replication environments .	4	2	8 (Low)	Federate all access using IAM Identity Center linked to your corporate Identity Provider (IdP), and mandate automated IAM credential inventory pruning.
Unpatched Software on Integration Hosts: Exploitation of known vulnerabilities within OS or third-party replication middleware hosted on 24/7 EC2 replication instances.	3	2	6 (Low)	Deploy AWS Systems Manager (SSM) Patch Manager to automate routine OS patching, and replace static hosts with fresh, hardened AMIs weekly.

Appendix D: Enterprise RACI Matrix

This matrix defines the clear operational boundaries, decision rights, and delivery expectations across cross-functional teams to eliminate role confusion and accelerate project execution.

Key Architectural Deliverable	Enterprise Architecture	SAP Basis / Infra (SAP)	Cloud / DevOps (Cloud)	Data & Analytics (Data)	CISO / Security (Sec)
Establishing Hybrid Network (Direct Connect / VPN)	A	C	R	I	C
Architecting Multi-Path Ingestion Strategy	A	C	C	R	C
Defining Data Cataloging & Lineage Strategy	A	I	C	R	C
Configuring SAP Communication Users & Least-Privilege Roles	I	R	I	C	A
Deploying Instance-Based Replication Middleware (SLT)	C	R	A	C	I
Provisioning Managed Connectors (AWS AppFlow, AWS Glue)	I	C	A	R	C
Configuring S3 Landing Zone, Buckets, & Lifecycle Policies	C	I	A	R	C
Scaling Target Environments (AWS Redshift & AWS RDS)	I	I	A	R	I
Building Data Quality & Dead Letter Queue (DLQ) Pipelines	I	I	C	R	A
Approving Cloud Risk Mitigation & Data-at-Rest Encryption	C	C	I	I	A
Managing AWS Secrets Manager & Rotating Target Credentials	I	C	A	R	C
Implementing Patch Management on EC2 Replication Hosts	I	C	R	I	A
Continuous Cloud Monitoring &	C	I	R	R	I

Operational Cost Tracking					
------------------------------	--	--	--	--	--

About the Author

Amit Kulkarni

Senior Solutions Architect

📍 Pune, India | ✉️ amitkwork2920@gmail.com | 🔗 [LinkedIn](#) | 💻 [Github](#)

I'm a Technology-agnostic architecture leader having 18+ years of partnership with 15+ clients across BFSI, Retail, Insurance, Healthcare, Travel, Media, and the Public Sector. Proven track record in cloud transformation (AWS, Azure), legacy COTS and mainframe modernization, security posture assessments, microservices modernization, event-driven architectures, and working knowledge on AI initiatives.

Architectural Specialization (Recent Focus)

During a recent deliberate transition period away from formal corporate employment, I dedicated my time to advanced deep-dives into enterprise cloud topologies, edge cases, and failure domains. This case study on *Legacy SAP Hybrid Modernization on AWS* is part of a broader body of architectural research aimed at critiquing standard vendor blueprints, identifying hidden system vulnerabilities, and building production-ready engineering remediations for high-throughput transactional environments.

I am currently seeking my next high-impact engineering role where I can solve complex distributed systems challenges and help scale robust, mission-critical infrastructure teams.